



Approximating the Shapley value with sampling : survey and new stratification techniques

François Lamothe, Sandra Ulrich Ngueveu

► To cite this version:

François Lamothe, Sandra Ulrich Ngueveu. Approximating the Shapley value with sampling : survey and new stratification techniques. 2025. <hal-04995993>

HAL Id: hal-04995993

<https://hal.science/hal-04995993v1>

Preprint submitted on 18 Mar 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons CC BY-SA 4.0 - Attribution - ShareAlike - International License

Approximating the Shapley value with sampling : survey and new stratification techniques

François Lamothe^{a,*}, Sandra Ulrich Ngueveu^a

^a*LAAS-CNRS, Université de Toulouse, INP, Toulouse, France*

Abstract

In game theory, a cooperative game can be used to model a system able to provide a service to a set of users for a certain cost. An important topic related to cooperative games is the cost sharing which distributes the total cost of the system among the players. Shapley proposed in 1953 a cost-sharing mechanism nowadays known as the Shapley value. It is usually interpreted as assigning to each player the cost this player induces on the system. This cost assignment has many desirable properties but it is very time consuming to compute exactly when the number of users is large. Thus, a large effort has been directed towards its approximation. Many general purpose algorithms for computing the Shapley value use sampling, and in particular stratified sampling, in order to obtain a good approximation. In this paper, we provide a comprehensive overview for approximating the Shapley value with sampling with a focus on how and why stratified sampling helps enhancing the precision of the approximation. We also propose a new paradigm to apply stratification which is more flexible than the currently used schemes and allows one to adjust the stratification decisions to the result of the sampling process. We suggest a methodology to create datasets containing a large number of random games to test Shapley approximation algorithms. Finally, we report an extensive experimental study of all the algorithms described in the paper on the datasets created using the previous methodology.

*Corresponding author

Email address: `flamothe@laas.fr` (François Lamothe)

Keywords: Game theory, Shapley value, Approximation, Sampling, Stratified Sampling

1. Introduction

In game theory, a cooperative game on a set of players N is represented by a function v mapping each subset S (also named coalition) of N with a real number $v(S)$. It can be used, for example, to model a system able to provide a service to any subset of the players. The real number $v(S)$ then represents the cost of servicing all players in S . An important topic related to cooperative games is cost sharing. A cost-sharing mechanism assigns a cost $\phi(v, i)$ to each player $i \in N$. It can represent how much player i should pay to benefit from the service. Shapley et al. (1953) proposed a cost-sharing mechanism nowadays known as the Shapley value. It is usually interpreted as assigning to each player the cost this player induces on the system. It satisfies many useful properties and in particular, the Shapley value is the only cost-sharing mechanism satisfying the four following axioms (Shapley et al., 1953).

- Efficiency : $\sum_{i \in N} \phi(v, i) = v(N)$;
- Anonymity : for any permutation π of N , $\phi(\pi(v), \pi(i)) = \phi(v, i)$ where $\pi(v)(S) = v(\{\pi(i), i \in S\})$;
- Dummy player : if for all $S \subset N$, $v(S) = v(S \cup \{i\})$ then $\phi(v, i) = 0$;
- Linearity : $\phi(v + w, i) = \phi(v, i) + \phi(w, i)$.

Unfortunately, the major drawback of this cost-sharing mechanism is that it is NP-Hard to compute (Deng & Papadimitriou, 1994) which means that it is very time consuming to compute exactly when the number of players is large. Thus a large effort has been directed toward its approximation. Many algorithms revolve around sampling random subsets S and evaluating the corresponding value $v(S)$. We give an overview of such algorithms in the next sections. The goal of such algorithms is to obtain the most precise

approximation of the Shapley value with a fixed number of evaluations of the value function v . Indeed, evaluating the value function is usually the most computationally intensive part of the algorithm. A technique used to enhance the precision of such algorithms without requiring additional samples is called stratification. This paper studies algorithms using sampling and stratification to approximate the Shapley value. We survey existing algorithms and propose new enhancements. In particular, our main contributions are the following.

- We provide a comprehensive overview for approximating the Shapley value with sampling. In particular, we focus on the use of stratification to enhance the quality of the approximation. We explain how and why stratification works and how it is used in the special case of the Shapley value.
- We propose a new paradigm to apply stratification which is more flexible than the currently used schemes. It allows to apply several types of stratification at the same time, which has not been done yet. Moreover, we propose to use a dynamic stratification method that allows one to adjust the stratification used to the result of the sampling process. Our experiments report, on several datasets, an increase in the quality of approximation generated by our algorithm compared to the ones from the literature. We also show how to determine if the modification we propose is going to be effective on a specific dataset.
- One important challenge when comparing estimators for the Shapley value comes in finding appropriate datasets. Indeed, one needs to find games whose Shapley value can be deduced quickly while still being non-trivial or satisfying specific properties. The literature so far used a small set of fixed games as baseline of comparison. We propose a methodology to derive a large number of random games satisfying certain properties. We also publish the datasets used in our experiments and the code used to create them.

The remainder of this paper is organized as follows. The next section sur-

veys the algorithms using sampling to approximate the Shapley value. Then, Section 3 contains a description of stratified sampling including how and why it enhances the approximation of the Shapley value. Section 4 describes the new stratification techniques that are the main contributions of this paper. It is followed in Section 5 with a description of the methodology that we propose to create datasets to test our algorithms. Finally, our experimental campaign is presented in Section 6 with the results and discussion.

2. Shapley value: formulas and approximation algorithms

In this section, we introduce several known formulae for the Shapley value. Each formula can be used to derive a basic algorithm which we will also present. In this article, we will denote n the cardinal of N , and s the cardinal of a subset S of N .

2.1. Marginal contributions formulae

The following formula is the most well-known definition of the Shapley value.

$$\phi(v, i) = \frac{1}{n!} \sum_{\pi \in \Pi(N)} v(S(\pi, i) \cup \{i\}) - v(S(\pi, i)) \quad (1)$$

where $\Pi(N)$ is the set of permutations of the players in N and $S(\pi, i)$ is the set of predecessors of i in permutation π .

In this formula, the Shapley value is defined as an average of the marginal contributions of player i over all possible permutations. The marginal contribution of a player i to a subset S is defined as $v(S \cup \{i\}) - v(S)$ and we will now denote it $\Delta^m(S, i)$. It represents the additional cost incurred by adding player i to the coalition S . In formula (1), the subset used in the marginal contribution is the set of predecessors $S(\pi, i)$ of player i in permutation π .

Another equally well-known formula based on marginal contributions is:

$$\phi(v, i) = \frac{1}{n} \sum_{S \subset N \setminus \{i\}} \frac{\Delta^m(S, i)}{\binom{n-1}{s}} \quad (2)$$

Both these formulae (1) and (2) can be interpreted as computing the expected marginal contribution of i to a random subset S of $N \setminus \{i\}$. This can be written as follows.

$$\phi(v, i) = \mathbb{E}[\Delta^m(S, i)]$$

In order for the expectation to be equal to the Shapley value, one must use the following distribution on the subsets S that we will call the Shapley distribution.

$$\text{(Shapley distribution) : } P(S) = \frac{1}{n \binom{n-1}{s}} \quad (3)$$

Both formulae (1) and (2) draw subsets following the above distribution (3) with two different processes.

Sampling method 1 : formula (1) draws a permutation π uniformly from $\Pi(N)$ and defines S to be $S(\pi, i)$.

Sampling method 2 : formula (2) selects a subset size uniformly over $\{0, \dots, n-1\}$ and then draws a subset S uniformly over the subsets of that size.

Finally, studying the multi-linear extension of the value v shows that one can derive a third formula where the subset S is drawn as follows (Okhrati & Lipani, 2021).

Sampling method 3 : draw a probability p uniformly from $[0, 1]$ then construct S by independently adding with probability p each player other than i to S .

Defining the Shapley value as an expected marginal contribution over subsets drawn in different manners puts under the same umbrella several sampling algorithms from the literature Castro, Gómez, and Tejada (2009); Castro, Gómez, Molina, and Tejada (2017); Okhrati and Lipani (2021). Indeed, following a classical idea to approximate expectations, one can approximate $\phi(v, i)$ by replacing the expectation with an average over a large set of randomly drawn subsets S_k . This leads to the following formula for the estimator.

$$\widehat{\phi(v, i)} = \frac{1}{K} \sum_{k=1}^K \Delta^m(S_k, i) \quad (4)$$

Remark on the sampling with permutations: Note, that if all manners of sampling lead to the same underlying distribution of the subsets S , the different algorithms should yield the same result. However, a special note can be made for the sampling method 1 *i.e.* sampling with permutations. After selecting a permutation, one can compute $v(S(\pi, i))$ for each player $i \in N$. This is enough to compute all marginal contributions since if j comes just after i in the permutation π then player i 's marginal contribution $v(S(\pi, i) \cup \{i\}) - v(S(\pi, i))$ equals $v(S(\pi, j)) - v(S(\pi, i))$. This means one can compute n marginal contributions with only n evaluations of the function v instead of $2n$ evaluations for the other methods (methods 2 and 3). As the number of evaluations is usually the limiting factor for approximating the Shapley value, the sampling by permutation has an edge over the other two methods.

Remark on the sampling with multi-linear extension: Note that Okhrati and Lipani (2021) report slightly better results with sampling with the multi-linear extension method than the one based on permutations. However, they give the same number of marginal contribution samples to each algorithm and not the same number of function evaluations; this negates the advantage of using permutation sampling discussed in the previous remark. Moreover, they do not exactly use Method 3. The method they use can be interpreted as including a form of proportional sample allocation (see Section 3.3) which is expected to increase the quality of the results.

Expected error of the approximation : A classical measure of the quality of an estimator $\widehat{\phi(v, i)}$ approximating a value $\phi(v, i)$ is its mean squared error $\mathbb{E}[(\widehat{\phi(v, i)} - \phi(v, i))^2]$. This represents how far away from the true value is usually the estimator.

The mean squared error of an estimator is known to be equal to its variance plus the square of its bias:

$$\mathbb{E}[(\widehat{\phi(v, i)} - \phi(v, i))^2] = \mathbb{V}[\widehat{\phi(v, i)}] + (\mathbb{E}[\widehat{\phi(v, i)}] - \phi(v, i))^2 \quad (5)$$

However, in the particular case of trying to approximate an expectation with a mean of sampled value, the estimator $\widehat{\phi(v, i)}$ is a unbiased; meaning

that $\mathbb{E}[\widehat{\phi(v, i)}] = \phi(v, i)$. This in turns means that the mean squared error of the estimator is equal to its variance. Let us now analyze the variance of the estimator. Recall that here $(S_k)_{k \in K}$ is a family of independently drawn random subsets of N .

$$\mathbb{V}[\widehat{\phi(v, i)}] = \mathbb{V}\left[\frac{1}{K} \sum_{k=1}^K \Delta^m(S_k, i)\right] \quad (6)$$

$$= \frac{1}{K^2} \sum_{k=1}^K \mathbb{V}[\Delta^m(S_k, i)] \quad (7)$$

$$= \frac{\mathbb{V}[\Delta^m(S, i)]}{K} \quad (8)$$

One can see that the main drivers of the variance and thus the mean squared error of the estimator are the variance of the random variable $\Delta^m(S, i)$ and the number of samples K . As a consequence, a game where the marginal contribution of a player varies little between subsets will have its Shapley value well estimated. Moreover, the higher the number of samples, the better the estimation.

2.2. Complementary contributions formulae

Another family of algorithms revolves around a different type of contributions called complementary contributions (Kolpaczki, Bengs, Muschalik, & Hüllermeier, 2024; Myerson, 1991; Zhang et al., 2023) By rearranging the terms in formula (2) and using the variable change $S' = N \setminus (S \cup \{i\})$ in one sum, one can express the Shapley value in terms of complementary contributions instead of marginal contributions (Zhang et al., 2023).

$$\begin{aligned}
\phi(v, i) &= \frac{1}{n} \sum_{S \subset N \setminus \{i\}} \frac{v(S \cup \{i\}) - v(S)}{\binom{n-1}{s}} \\
&= \frac{1}{n} \sum_{S \subset N \setminus \{i\}} \frac{v(S \cup \{i\})}{\binom{n-1}{s}} - \frac{1}{n} \sum_{S \subset N \setminus \{i\}} \frac{v(S)}{\binom{n-1}{s}} \\
&= \frac{1}{n} \sum_{S \subset N \setminus \{i\}} \frac{v(S \cup \{i\})}{\binom{n-1}{s}} - \frac{1}{n} \sum_{S \subset N \setminus \{i\}} \frac{v(N \setminus (S \cup \{i\}))}{\binom{n-1}{s}} \\
&= \frac{1}{n} \sum_{S \subset N \setminus \{i\}} \frac{v(S \cup \{i\}) - v(N \setminus (S \cup \{i\}))}{\binom{n-1}{s}} \\
&= \frac{1}{n} \sum_{S \subset N \setminus \{i\}} \frac{v(S) - v(N \setminus S)}{\binom{n-1}{s-1}} \\
&= \frac{1}{n} \sum_{S \subset N \setminus \{i\}} \frac{\Delta^c(S)}{\binom{n-1}{s-1}} \tag{9}
\end{aligned}$$

This formula can also be interpreted as the expected complementary contribution $\Delta^c(S) = v(S) - v(N \setminus S)$ of random subsets:

$$\phi(v, i) = \mathbb{E}[\Delta^c(S)] \tag{10}$$

To obtain the appropriate underlying distribution, draw a subset size uniformly from $\{0, \dots, n-1\}$ then draw a subset uniformly among subsets of $N \setminus \{i\}$ of that size, and finally add player i to it. Note that the subset obtained before the addition of player i follows the Shapley distribution (3). Because of this expectation interpretation, one can again design an algorithm by computing an average of random samples.

This reformulation was developed for the following reason. A complementary contribution $\Delta^c(S)$ appears in the sum (9) of all players in S while $-\Delta^c(S)$ appears in the sum of all players in $N \setminus S$. Zhang et al. (2023) propose to draw a subset size uniformly from $\{1, \dots, n\}$ then draw a subset uniformly among subsets of that size. Then, the computed complementary contribution (which costs two evaluations of the value function) can be used to update the Shapley estimates of all players. Because each complementary contribution can be used to update the average of several players, each

average contains more samples for the same number of evaluations of the value function v . This is a large gain in efficiency because the precision of the estimate is directly related to number of terms in the average.

However, this method to reuse samples comes with some drawbacks. First, when reusing the contributions in this way, the sampled contributions will never follow the distribution used in the expectation formula (10). Indeed, whatever the sampling distribution used, every time a complementary contribution $\Delta^c(S)$ is sampled and computed, it is used to update the estimate of s players while $-\Delta^c(S)$ is used to update the other $n - s$ players. Note that $-\Delta^c(S)$ is also the complementary contribution corresponding to the $n - s$ -sized subset $N \setminus S$. This means the final average will not contain the same number of samples corresponding to each (which is the case in the distribution used in the expectation formula). In particular, the average will contain $n - 1$ times more terms corresponding to subsets of size $n - 1$ than terms corresponding to subsets of size 1. This means that using a simple average while reusing the complementary contributions for each player would not yield a properly converging estimate of the Shapley value (when the number of samples increases). However, this can be alleviated. Using stratification (see Section 3), one can reuse the complementary contribution for all players and still obtain a converging estimate of the Shapley value. Nevertheless, since small sizes will receive few samples, their contribution to the average will not be very well estimated. This again can be partially alleviated by sample allocation (see Section 3.3) and explains why sample allocation has a large positive impact when used with complementary contributions.

Expected error of the approximation : Similarly to the marginal contribution approximation, the approximation used in this section estimates the expectation of a random variable with an average of sampled values. Thus again, the mean squared error of the estimator is equal to the variance of

the estimator which is equal to:

$$\mathbb{V}[\widehat{\phi(v, i)}] = \mathbb{V}\left[\frac{1}{K} \sum_{k=1}^K \Delta^c(S_k)\right] \quad (11)$$

$$= \frac{1}{K^2} \sum_{k=1}^K \mathbb{V}[\Delta^c(S, i)] \quad (12)$$

$$= \frac{\mathbb{V}[\Delta^c(S)]}{K} \quad (13)$$

Thus, the error of the estimator is again driven by two factors: the number of samples and the variance of the random variable $\Delta^c(S)$. Thus, compared to an estimator based on marginal contribution, there are two differences. Because one can reuse samples from complementary contributions, there will be a higher number of terms in the mean for the same number of value function evaluations. However, the variance of complementary contributions is usually higher than the variance of marginal contributions. Thus, a method based on complementary contributions will work better than one based on marginal contributions when the increase in variance is largely compensated by the sample reuse. **In practice, complementary contributions work best when both variances are similar. On the contrary, marginal contributions work best when the difference in variance is high.** This really depends on the set of instances considered. This will be illustrated in our numerical results in Section 6.3.

2.3. Quadratic program

Another method to obtain the Shapley value is to define it as the solution to a quadratic optimization problem. This method has led to several approximation algorithms for the Shapley value, like KernelSHAP which is rather popular in the machine learning community. Using a variable x_i to represent $\phi(v, i)$, the Shapley value can be computed as the solution of the

following quadratic program.

$$\begin{aligned} \min \quad & \sum_{S \subsetneq N, S \neq \emptyset} \binom{n}{s}^{-1} \frac{n-1}{s(n-s)} \left(\sum_{i \in S} x_i - v(S) \right)^2 \\ \text{s.t.} \quad & \sum_{i \in N} x_i = v(N) \end{aligned}$$

One can approximate the Shapley value by drawing random subsets with a probability proportional to $\binom{n}{s}^{-1} \frac{n-1}{s(n-s)}$ leading to a family $\mathcal{S}$ of subsets. Then the following quadratic program is solved:

$$\begin{aligned} \min_{\phi} \quad & \sum_{S \in \mathcal{S}} \left(\sum_{i \in S} x_i - v(S) \right)^2 \\ \text{s.t.} \quad & \sum_{i \in N} x_i = v(N) \end{aligned}$$

Although, this algorithm also requires sampling, it does not involve approximating an expectation. Our preliminary numerical experiments show that it does not benefit from stratification.

In the above section, we have presented the main algorithms using sampling to approximate the Shapley value of a cooperative game. In the next section, we will focus on stratified sampling, a statistical technic enabling the sampling process to converge faster and deliver a better approximation.

3. Stratified sampling

Several techniques can be used to enhance the effectiveness of a sampling process without requiring additional samples like kernel herding (Mitchell, Cooper, Frank, & Holmes, 2022) and antithetic sampling (Staudacher & Pollmann, 2023). In this work we will focus on stratified sampling, a technic broadly used to improve approximation algorithm for the Shapley value (Castro et al., 2017; Castro, Gómez, & Tejada, 2009; Kolpaczki et al., 2024; Maleki, Tran-Thanh, Hines, Rahwan, & Rogers, 2014; Zhang et al., 2023).

The goal of stratified sampling is to remove the bias introduced in the sampling process when the sample set does not perfectly match the sampling

distribution. The idea is the following. Let us assume an expectation $\mathbb{E}[X]$ is being approximated by sampling using

$$\mathbb{E}[X] \approx \hat{X} = \frac{1}{|\Sigma|} \sum_{\sigma \in \Sigma} x_{\sigma} \quad (14)$$

where $\sigma \in \Sigma$ denotes a sample. Here, x_{σ} represents the value of the sample σ . We will denote X_{σ} the random variable modeling x_{σ} before the sampling process occurred.

Let us assume that we know the probability $P(A)$ of an event A (we will denote its complement $\bar{A}$) and that for each sample x_k we can say whether event A occurred during the sampling of x_k . For example, when sampling marginal contributions we know that the event "the drawn subset S has size 3" happens with probability $\frac{1}{n}$ and we can trace whether or not a particular sampled marginal contribution was made using a subset of size 3. In this work, we will say that such a sample "corresponds" to the event A .

With this knowledge, we can create a better estimation of $\mathbb{E}[X]$ by using the law of total expectation:

$$\mathbb{E}[X] = \mathbb{E}[X|A]P(A) + \mathbb{E}[X|\bar{A}]P(\bar{A}) \quad (15)$$

We can divide the samples into two groups $(x_{\sigma})_{\sigma \in \Sigma_A}$ and $(x_{\sigma})_{\sigma \in \Sigma_{\bar{A}}}$: those corresponding to A and those corresponding to $\bar{A}$. Then we can approximate the two conditional expectations separately and obtain the following estimate of $\mathbb{E}[X]$.

$$\hat{X}_{st} = \frac{P(A)}{|\Sigma_A|} \sum_{\sigma \in \Sigma_A} x_{\sigma} + \frac{P(\bar{A})}{|\Sigma_{\bar{A}}|} \sum_{\sigma \in \Sigma_{\bar{A}}} x_{\sigma} \quad (16)$$

This new estimator is called the stratified estimator and is usually a better approximation of the true value $\mathbb{E}[X]$.

Note that the above discussion generalizes to the context where one knows a partition of events $A_1, \dots, A_K$ and their corresponding probabilities. This leads to the following formula.

$$\hat{X}_{st} = \sum_{k=1}^K \frac{P(A_k)}{|\Sigma_{A_k}|} \sum_{\sigma \in \Sigma_{A_k}} x_{\sigma} \quad (17)$$

3.1. Why does stratification improve the quality of an estimator?

A classical measure of the quality of an estimator is its mean squared error $\mathbb{E}[(\hat{X} - \mathbb{E}[X])^2]$ which is known to be equal to its variance plus the square of its bias.

$$\mathbb{E}[(\hat{X} - \mathbb{E}[X])^2] = \mathbb{V}[\hat{X}] + (\mathbb{E}[\hat{X}] - \mathbb{E}[X])^2 \quad (18)$$

Let us assume that the sizes $|\Sigma_A|$ and $|\Sigma_{\bar{A}}|$ are known (which is true at the end of the sampling). Then, denoting $Q_A = 1 - Q_{\bar{A}} = \frac{|\Sigma_A|}{|\Sigma|}$, the variance of both estimators (14) and (16) are:

$$\begin{aligned} \mathbb{V}[\hat{X}] &= \mathbb{V} \left[\frac{1}{|\Sigma|} \sum_{\sigma \in \Sigma} X_{\sigma} \right] \\ &= \frac{1}{|\Sigma|^2} \sum_{\sigma \in \Sigma} \mathbb{V}[X_{\sigma}] \\ &= \frac{|\Sigma_A| \mathbb{V}[X|A] + |\Sigma_{\bar{A}}| \mathbb{V}[X|\bar{A}]}{|\Sigma|^2} \\ &= \frac{Q_A \mathbb{V}[X|A] + Q_{\bar{A}} \mathbb{V}[X|\bar{A}]}{|\Sigma|} \end{aligned}$$

and

$$\begin{aligned}
\mathbb{V}[\hat{X}_{st}] &= \mathbb{V} \left[\frac{P(A)}{|\Sigma_A|} \sum_{\sigma \in \Sigma_A} X_\sigma + \frac{P(\bar{A})}{|\Sigma_{\bar{A}}|} \sum_{\sigma \in \Sigma_{\bar{A}}} X_\sigma \right] \\
&= \frac{P(A)^2}{|\Sigma_A|^2} \sum_{\sigma \in \Sigma_A} \mathbb{V}[X|A] + \frac{P(\bar{A})^2}{|\Sigma_{\bar{A}}|^2} \sum_{\sigma \in \Sigma_{\bar{A}}} \mathbb{V}[X|\bar{A}] \\
&= \frac{\frac{P(A)^2}{Q_A} \mathbb{V}[X|A] + \frac{P(\bar{A})^2}{Q_{\bar{A}}} \mathbb{V}[X|\bar{A}]}{|\Sigma|}
\end{aligned}$$

The weak law of large numbers tells us that Q_A converges (in probability) toward $P(A)$ (and $Q_{\bar{A}} \rightarrow P(\bar{A})$) when the number of samples grows. Thus, both $\mathbb{V}[\hat{X}]|\Sigma|$ and $\mathbb{V}[\hat{X}_{st}]|\Sigma|$ converge to $P(A)\mathbb{V}[X|A] + P(\bar{A})\mathbb{V}[X|\bar{A}]$. Thus, for a sufficiently large number of samples, we can assume that the variance of both estimators is equal.

Meanwhile, by linearity of the expectation we have $\mathbb{E}[\hat{X}_{st}] = \mathbb{E}[X]$ which means the estimator $\hat{X}_{st}$ is unbiased. On the contrary, knowing the size of $|\Sigma_A|$, the expectation of $\hat{X}$ is equal to $Q_A\mathbb{E}[X|A] + Q_{\bar{A}}\mathbb{E}[X|\bar{A}]$ which is equal to $\mathbb{E}[X]$ only when $Q_A = P(A)$ (see equation (15)).

Overall, we can see that stratification corrects the fact that the sampling process does not always create the number of samples expected to belong to an event A , *i.e.* $Q_A \neq P(A)$. More precisely, stratification does not change the variance of the estimator but corrects the bias introduced by this incorrect number of samples.

3.2. Stratification event

Once the principles of stratified sampling have been laid down, one has to decide which event A (or which partition of events $A_1 \dots A_K$) to use in the context of the Shapley value.

The event partition most commonly used in the literature (Castro et al., 2017; Maleki et al., 2014; Zhang et al., 2023) is on the size of the drawn subset. More precisely, when drawing subsets S for computing the marginal contributions of player i , we are interested in the event " $|S| = s$ " for $s \in$

$\{0, \dots, n-1\}$. We know that in the Shapley distribution (3), each of these events has probability $\frac{1}{n}$. This can be seen by recognizing that in an n -player random permutation, the probability of player i being in any specific position is $\frac{1}{n}$.

Another event partition proposed by Maleki (2015) is the presence (or not) of another player j in the subset : " $j \in S$ " or " $j \notin S$ ". In fact, this can be expended to the presence of more than one player. For example for two players, " $j \in S$ and $k \in S$ " or " $j \in S$ and $k \notin S$ " or " $j \notin S$ and $k \in S$ " or " $j \notin S$ and $k \notin S$ ". The following proposition gives the probability of these events happening.

Proposition 1. *Let $i_1, \dots, i_p$ and $j_1, \dots, j_q$ be some players of $N \setminus \{i\}$. Then under the Shapley distribution (3),*

$$P("i_1, \dots, i_p \in S \text{ and } j_1, \dots, j_q \notin S") = \frac{p!q!}{(p+q+1)!}$$

Proof. Under the Shapley distribution (3), the probability $P("i_1, \dots, i_p \in S \text{ and } j_1, \dots, j_q \notin S")$ is equal to the probability that in a random permutation, $i_1, \dots, i_p$ come before i while $j_1, \dots, j_q$ come after.

Let us consider the following way to construct a permutation. First, consider player i alone in the permutation. Then, insert player i_1 uniformly in any of the two possible positions (before or after i). Then, insert player i_2 uniformly in any of the three possible positions (before i and i_1 , between i and i_1 or after i and i_1). Continue inserting the players uniformly into the permutation until all players have been added. Note that this construction yields a uniform distribution over the set of permutations. This means the set of predecessors of player i in this construction follows a Shapley distribution (3).

Let us now consider that we insert first player i , then $i_1, \dots, i_p$, then $j_1, \dots, j_q$ and finally the other players. Whether the event " $i_1, \dots, i_p$ come before i while $j_1, \dots, j_q$ come after" occurs depends only on where the players $i_1, \dots, i_p$ and $j_1, \dots, j_q$ are inserted and not on where the other players are inserted. Also note that, the probability of inserting a player j before player i when there are already p players before i and q players after is $\frac{p+1}{p+q+2}$. Meanwhile, the probability of inserting player j after i in the same context is $\frac{q+1}{p+q+2}$. In the end, by computing the product of probabilities of the insertions of $i_1, \dots, i_p$, then $j_1, \dots, j_q$ we obtain a probability for the studied event of :

$$\frac{1}{2} \frac{2}{3} \dots \frac{p}{p+1} \frac{1}{p+2} \dots \frac{q}{p+q+1}$$

This is equal to the announced probability. $\square$

Remark - Future work: The size of the subset S is also the size of $S \cap N$. Moreover, a player j is present in a subset S if and only if the size of $S \cap \{j\}$ equals one. This means that all events used for stratification in the literature are of the form " $|S \cap E| = k$ " for E being the complete set N or a singleton. One could generalize this to any subset $E \subset N$. Although we did not investigate this in our work, this may be a promising avenue for future work.

3.3. Sample allocation

For certain events, it is possible to create samples specifically corresponding to an event or not. For example, it is possible to sample subsets with a specific size k . When this is possible, one can decide the number of samples allocated to each event A . This is called sample allocation and is used by many authors (Castro et al., 2017; Kolpaczki et al., 2024; Zhang et al., 2023). Let us consider, a stratification according to an event A . We can consider three cases of sample allocation.

No sample allocation : Left alone, the sampling process will draw a proportion of samples corresponding to event A that will converge towards $P(A)$ when the number of samples is large. In this case, stratification can be used to ensure the weight of the samples corresponding to event A in the average equals $P(A)$.

Proportional sample allocation : In this case, one allocates a proportion of samples corresponding to event A equal to $P(A)$. In this case, using stratification has no further impact.

Neyman sample allocation : this allocation was created to maximize the precision of the stratified estimator (16) by minimizing its mean squared error. Recall from Section 3 that the mean squared error of the stratified estimator is equal to its variance which is equal to :

$$\mathbb{V}[\hat{X}_{st}] = \frac{\frac{P(A)^2}{Q_A} \mathbb{V}[X|A] + \frac{P(\bar{A})^2}{Q_{\bar{A}}} \mathbb{V}[X|\bar{A}]}{|\Sigma|} \quad (19)$$

$$= \frac{P(A)^2}{|\Sigma_A|} \mathbb{V}[X|A] + \frac{P(\bar{A})^2}{|\Sigma_{\bar{A}}|} \mathbb{V}[X|\bar{A}] \quad (20)$$

If one can control the sample allocation, one can decide the values of $|\Sigma_A|$ and $|\Sigma_{\bar{A}}|$. One can minimize the function $|\Sigma_A|, |\Sigma_{\bar{A}}| \rightarrow \mathbb{V}[\hat{X}_{st}]$ under a fixed budget of samples : *i.e.* $|\Sigma_A| + |\Sigma_{\bar{A}}| = Cste$. Doing this for an event partition $(A_k)_{k \in K}$ leads to the following Neyman allocation (see the work of Castro et al. (2017) for more details).

Proposition 2. *Let us assume a given sample budget B and an event stratification $(A_k)_{1 \leq k \leq K}$. Then the number of samples x_k to allocate to the strata corresponding to A_k in order to minimize the variance of the estimator (16) should be proportional to $P(A_k)\sqrt{\mathbb{V}[X|A_k]}$:*

$$x_k = \frac{c_k B}{\sum_{1 \leq k \leq K} c_k}$$

where $c_k = P(A_k)\sqrt{\mathbb{V}[X|A_k]}$.

Proof. See in Appendix A □

In order to perform the Neyman allocation, one requires the values $\mathbb{V}[X|A]$ and $\mathbb{V}[X|\bar{A}]$. Thus, it is classical to run the algorithms using the Neyman allocation in two phases. In phase one, samples are drawn for both events A and $\bar{A}$ (without sample allocation or with proportional sample allocation). These samples are used to estimate $\mathbb{V}[X|A]$ and $\mathbb{V}[X|\bar{A}]$. Then, in the second phase, the previous samples are completed in order to obtain a distribution of samples proportional to $P(A)\sqrt{\mathbb{V}[X|A]}$ and $P(\bar{A})\sqrt{\mathbb{V}[X|\bar{A}]}$. Note that in the Neyman allocation it is required to use the stratified estimator in order to obtain a properly converging estimator of the Shapley value.

The case of sample allocation for estimator based on complementary contributions is slightly more convoluted and we refer the reader to (Zhang et al., 2023) for more details. There are other ways to allocate the samples. For example, Burgess and Chapman (2021) proposed to allocate the samples based on a Bernstein probabilistic bound on the error of the estimator.

4. A more flexible stratification paradigm

In this section, we present the main contribution of this paper; that is a more flexible and powerful paradigm to apply stratification.

One of the weaknesses of the stratification method presented in Section 3 is that one can only stratify according to one event partition. If given two partition events $A, \bar{A}$ and $B, \bar{B}$ one can merge them into the following partition : $A \cap B, A \cap \bar{B}, \bar{A} \cap B$ or $\bar{A} \cap \bar{B}$. However, this raises two questions. One, do we know the probabilities of these intersections of events? Two, if given K event partitions to merge, the number of intersection events becomes 2^K which becomes quickly intractable.

Another weakness of classical stratification, is that it does not react to the result of the sampling process; meaning the event partition is fixed in advance. Let us assume that we are stratifying according to an event A . If the sampling process turns out, by chance, to allocate the samples proportionally to $P(A)$ and $P(\bar{A})$ then stratifying according to A will have no impact. One may say that it does not hurt either. However, if we have more possible stratification events than we can use (which is the case for the Shapley value), we may want to focus on those which matter.

We address the two above weaknesses in the next two sections.

4.1. Stratification as an optimization problem

One of the main effects of stratification (if we exclude sample allocation strategies) is to ensure that the total weight of the samples corresponding to an event A is equal to $P(A)$. This ensures that the sampling is more representative of the distribution underlying the expectation being estimated. To formalize this idea, we express the weight of each sample as a variable of an optimization problem.

Let us denote $y = (y_\sigma)_{\sigma \in \Sigma}$ the vector modeling the weight of each sample $\sigma \in \Sigma$. Let us also consider $(A_k)_{1 \leq k \leq K}$ a family of events used for stratification and denote $\mathbb{1}_{A_k}$ the vector of size $|\Sigma|$ indicating for each sample whether or not it belongs to event A_k . This enables us to compute the sum of weights of the samples corresponding to an event A_k with $\mathbb{1}_{A_k}^T y$. Then, to ensure that the sum of weight of the samples corresponding to one of the events A_k is equal to $P(A_k)$, one can enforce the following set of constraints.

$$\forall 1 \leq k \leq K, \quad \mathbb{1}_{A_k}^T y = P(A_k) \quad (21)$$

However, if there are more events than samples, this linear system may have no solution. Thus, in order to solve this issue, we propose to use the following quadratic program.

$$\min_y \sum_{k=1}^K \alpha_k (\mathbb{1}_{A_k}^T y - P(A_k))^2 \quad (22)$$

The objective function softly enforces constraints (21) with a quadratic penalization. Also note the presence of coefficients α_k whose role is to decide the strength of each quadratic penalization.

Benefit : The main advantage of this method of stratification is its flexibility. Indeed, one can exploit with it any number of events A_k as long as the probability $P(A_k)$ is known. In particular, the events $(A_k)_{1 \leq k \leq K}$ do not need to be mutually disjoint as in an event partition. In particular, one can stratify according to two or more event partitions at the same time. For example, in our case we will want to stratify both according to the subset size and the presence of other players as discussed in Section 3.2.

Solving problem (22), method 1: By developing the square in the objective function, one can put the quadratic program (22) under the form $\min y^T A y + b^T y + c$ by setting the following.

$$A = \sum_{k=1}^K \alpha_k \mathbb{1}_{A_k} \mathbb{1}_{A_k}^T \quad (23)$$

$$b = \sum_{k=1}^K \alpha_k \mathbb{1}_{A_k} \quad (24)$$

$$c = \sum_{k=1}^K \alpha_k^2 P(A_k)^2 \quad (25)$$

The solution of the quadratic program is then $A^{-1}b$. Since the matrix A is the sum of K matrices of size $|\Sigma|$, the computing time of the creation of A depends on the number of events and on the number of samples. However, the computing time of the inversion only depends on the number of samples. This method is effective when the number of samples is not too high; otherwise the matrix A is too big to handle.

Solving problem (22), method 2: when there are less events than samples, the linear system (21) has a solution. One could solve it directly; although it does not have a unique solution. However, it is also possible to obtain a solution by the following iterative approach, starting from a vector y equal to the all one vector. One can enforce equation $\mathbb{1}_{A_k} y = P(A_k)$ by uniformly multiplying by a factor $\frac{P(A_k)}{\mathbb{1}_{A_k} y}$ the coefficients y_σ of the samples σ corresponding to event A_k . This multiplication can be applied sequentially for all the events A_k . After several rotations over all events, the vector y will converge toward a solution of the linear system (21). This convergence is rather quick. This method is named *raking* in the statistics literature.

Solving problem (22), method 3: when there are more events than samples we cannot use the iterative method of the previous paragraph because it does not converge. However, a solution of problem (22) also satisfies the following fixed point equation.

$$y = \frac{\sum_{k \in K} \beta_k P_{A_k}(y)}{\sum_{k \in K} \beta_k} \quad (26)$$

where $\beta_k = \alpha_k \mathbb{1}_{A_k}^T \mathbb{1}_{A_k}$ and $P_{A_k}(y)$ is the orthogonal projection of y onto the hyperplane defined by the equation $\mathbb{1}_{A_k} y = P(A_k)$. The point $P_{A_k}(y)$ is also the closest point to y that satisfies the equation $\mathbb{1}_{A_k} y = P(A_k)$. Because there are too many events, not all equations $\mathbb{1}_{A_k} y = P(A_k)$ can be satisfied at the same time. Thus, intuitively, a solution of problem (22) becomes an average of points that each satisfy one equation. This average is weighted by the coefficients β_k to take into account the weight α_k and the fact that the vector $\mathbb{1}_{A_k}$ are not normalized. One can iterate of this equation by starting from any point y_0 and iterating over :

$$y_{i+1} = \frac{\sum_{k \in K} \beta_k P_{A_k}(y_i)}{\sum_{k \in K} \beta_k} \quad (27)$$

This iterative method does not converge as fast as the previous one but works for any number of events.

4.2. Dynamic stratification and choice of the stratification events

In this section, we assume having a list of events $A_1, \dots, A_K$ for which we can compute the probability of any intersection of these events or their

complement *e.g.* $A_1 \cap \bar{A}_2 \cap A_4$. For example, we can compute these probabilities with Proposition 1 for events of the $j \in S$ as presented in Section 3.2. However, completely merging all these events into one event partition would lead to a partition with 2^K events which is not tractable.

To solve this issue, we introduce a new method that greedily creates an event partition from the list of events $A_1, \dots, A_K$. Moreover, this method is applied after the sampling was made so that the partition takes into account the result of the sampling process.

When an expectation $\mathbb{E}[X]$ is estimated with an average, one can enhance the precision of the estimation by stratifying according to an event A by using the total expectation formula,

$$\mathbb{E}[X] = P(A)\mathbb{E}[X|A] + P(\bar{A})\mathbb{E}[X|\bar{A}] \quad (28)$$

and estimating separately $\mathbb{E}[X|A]$ and $\mathbb{E}[X|\bar{A}]$ with averages. The idea in this section is that we can recursively apply the total expectation formula to the two expectations $\mathbb{E}[X|A]$ and $\mathbb{E}[X|\bar{A}]$. For example, $\mathbb{E}[X|A]$ can be further decomposed with

$$\mathbb{E}[X|A] = P(B|A)\mathbb{E}[X|A \cap B] + P(\bar{B}|A)\mathbb{E}[X|A \cap \bar{B}]$$

which leads to two new expectations to estimate. And these new expectations can also be decomposed again. With this process we are creating an increasingly large event partition with which to stratify. Each time we decompose an expectation with the total expectation formula, we need to choose an conditioning event from the list $A_1, \dots, A_K$ of candidate events. To that end, we estimate according to some metric the expected gain in precision achieved by stratifying using each event A_k in the list. Then, we stratify using the most promising event.

A difficulty with this method is to find a good metric to estimate which events are promising. As detailed in Section 3.1, the main reason why stratification improves the quality of the estimator is that it removes the bias introduced by the sampling process. Indeed, the estimator is biased when the number of samples corresponding to an event A_k is not proportional to $P(A_k)$. Thus, we compute the bias of the non-stratified estimator to estimate

the improvement gained from stratification. The event with the highest bias is chosen (so that the highest bias is removed through stratification).

Remark : Note that there is a limit to recursive stratification. Indeed, after p recursive stratifications one has to estimate an expectation of the form $\mathbb{E}[X|A_{k_1}, \dots, A_{k_p}]$ with an average. The available samples corresponding to the event $A_{k_1} \cap \dots \cap A_{k_p}$ may be in very low number; in the extreme case, there could be none. Estimating the expectation with a very low number of samples will yield an estimation of poor quality. Thus, one should stop the recursive stratification when the number of samples remaining to estimate an expectation becomes low (in our experiments, we set this bound on the number of remaining samples equal to the number of players).

Remark : It is possible to combine the recursive stratification with the formulation of stratification as an optimization problem. Let us assume that a stratification event A is being recursively cut into $A \cap B / A \cap \overline{B}$. One can at the same time cut the event A into $A \cap C / A \cap \overline{C}$. This creates four events instead of two and all these events can be added to the optimization problem (or further recursively cut). For example, this means that when deciding which events to keep in the list of candidate events $A_1, \dots, A_K$ we can select more than one to apply stratification. However, note that choosing more than one at every recursive stratification step grows the number of considered events very quickly. Furthermore, our preliminary tests did not show an additional gain in precision from selecting more than one event.

5. Datasets for evaluating Shapley value estimators

In this section, we present another contribution of this paper, answering an important challenge when comparing estimators for the Shapley value : finding appropriate datasets.

5.1. Why do we need new datasets ?

A good dataset would preferably meet the following properties.

- It contains games whose value function and Shapley value can be evaluated efficiently.

- The games in the dataset should be representative of real world games. Or at least there should be games with many different properties and behaviors.
- The games should be created with some form of random procedural generation in order to be able to create large datasets and assess the quality of the compared estimators using averages and other statistical tools.

So far, the literature used a small set of fixed games as baseline of comparison. These games would arguably satisfy the first two properties. However, because their set is composed of less than 10 instances, it is difficult to perform a meaningful statistical analysis. In this section, we discuss how to create datasets satisfying all these properties. Note that the datasets obtained from our method can be inspired from the classical instances from literature to retain their properties. In particular, we create datasets corresponding to voting games and airport games (Castro, Gómez, & Tejada, 2009).

5.2. Sum of symmetric games

In this section, we describe a useful family of games to create datasets for assessing the quality of Shapley estimators. This family is composed of all games that are the sum of symmetric games. Using the notation $\pi(S) = \{\pi(i) | \forall i \in S\}$ let us define symmetric games.

Definition 1. *A game v is said to be symmetric if for all permutations π where dummy players are invariant:*

$$v(S) = v(\pi(S))$$

For this definition recall that dummy players i are such that $v(S) = v(S \cup \{i\})$ for all $S \subset N$. In a less formal language, a symmetric game is a game where all non-dummy players are identical. Then, thanks to the definition of symmetric games, we have that all subsets with the same number of non-dummy players have the same value.

One can define a game v as a sum of symmetric games in the following way.

- $v = \sum_i v^i$ where each v^i is a symmetric game
- each v^i has its set of dummy players D_i
- $v^i(S) = v_s^i$ where s is the number of non-dummy players in S ; this requires defining only $n - |D_i|$ values.

Value function evaluation: To evaluate a subset S in a symmetric game v , one only needs to compute the number s of non-dummy players in S and return v_s . For a sum of symmetric games, one can sum the evaluation obtain for each symmetric game.

Shapley value evaluation: First, the axiom of linearity ensures that the Shapley value of a sum of symmetric games is the sum of the Shapley values of each symmetric game. Moreover, by combining the definition of symmetry with the axioms of efficiency, anonymity and dummy players of the Shapley value, one can compute the Shapley value of a symmetric game v where the set of dummy players is D as follows. For all non-dummy players $i \notin D$, $\phi(v, i) = \frac{v(N)}{n - |D|}$ otherwise $\phi(v, i) = 0$.

The class of sums of symmetric games is very practical to create datasets. Indeed, the value function and the Shapley value can be efficiently evaluated. The procedural generation only requires, for each symmetric game, the choice of a subset of non-dummy players and a function $s \rightarrow v_s$. Note that a sum of symmetric games is no longer a symmetric game. In fact, one can decompose any game v as a weighted sum of the following symmetric games. For each $S \subset N$ consider the game v^S whose value is: $v^S(S') = v(S)$ if $S \subset S'$ otherwise 0. However, note that this decomposition requires an exponential number (in n) of symmetric games to work. One of the main advantage of this class of games is that if the functions $s \rightarrow v_s$ of all the symmetric games are sub-additive/super-additive/sub-modular/super-modular/monotone then the same property applies to complete game (the sum of the symmetric games). Therefore, it is relatively easy to generate, for example, a dataset of random sub-additive games.

6. Numerical results

In this section we present the numerical results obtained during this work. The datasets and the Julia code used in this section can be accessed at https://github.com/TwistedNerves/shapley_approximation.

In the previous sections of this work we have presented several algorithms with their variants and have proposed a theoretical analysis for each of them. In this section we want to empirically validate the previous analysis with computational results. We will thus answer the following questions in our numerical results.

- When should one use an algorithm based on complementary contributions instead of an algorithm based on marginal contribution?
- How effective is stratification : according to player position? According to the presence of other players using the recursive strategy presented in Section 4.2?
- How effective is it to use both stratifications and to be able to combine them using the flexible stratification paradigm presented in Section 4.1?
- When is the Neyman sample allocation presented in Section 3.3 effective?
- When is it relevant to use the algorithm based on the quadratic program presented in Section 2.3 ?
- Overall, which algorithms appears do dominate the others ?

This section is dedicated to presenting the datasets used, the algorithms tested and then our results.

6.1. Used datasets

All datasets used for our numerical study will be based on the paradigm of sum of symmetric games presented in Section 5.2. Each dataset is composed of 4×20 games with 20 games for each number of players : $\{10, 20, 50, 100\}$.

Unless mentioned otherwise, each game is the sum of $5n$ symmetric games where n is the number of players. Each symmetric game contains a random number of non-dummy player varying from 2 to n . This number of non-dummy is set to $\exp(X)$ where X follows a uniform distribution between $\log(2)$ and $\log(n)$. Each symmetric game v is normalized with $v(N) = 1$ so that each symmetric game contributes the same amount to the Shapley value. To describe a dataset there mainly remains to describe the function $s \rightarrow v_s$ associating each number of non-dummy player with a value.

The datasets are then the following :

Dataset random: this dataset is constructed as described above with v_s being randomly chosen from a uniform distribution $U(0, 1)$.

Dataset random sparse: similar to the previous dataset; except that the number of non-dummy players of each symmetric game has a probability $\frac{3}{4}$ of being 2 and $\frac{1}{4}$ of being set as described before. This is done in order to generate many games with two non-dummy players. In a such games, if the two non-dummy players are i and j , the presence of j in a subset S has a large influence on the marginal contribution $\Delta^m(S, i)$. The *sparse* datasets are used to simulate datasets where the presence of a specific player in a coalition has a large impact on the marginal cost of another player in this coalition. This type of behavior can be found in instances of the literature such as the *minimum spanning tree* instance (Zhang et al., 2023) where neighbor nodes are linked in this manner.

Dataset monotone: in this dataset, $v_1 = 1$ and $v_{s+1} = v_s + X$ where X follows a uniform distribution $U(0, 1)$. This results in the function $s \rightarrow v_s$ being monotone increasing.

Dataset monotone sparse: similar to the dataset *monotone* except that the number of non-dummy players is chosen as in the dataset *random sparse*.

Dataset voting: voting games are classical in game theory and this dataset is inspired from a classically used instance from the literature (Castro, Gómez, & Tejada, 2009). They simulate an assembly. The value of a coalition is one if it has enough voting power to pass a law and zero otherwise. This dataset is composed of symmetric voting games which means all

non-dummy players have the same voting power whereas the dummy players have a voting power of zero. Each voting game is composed of one symmetric game. Its number of non-dummy player is set to $\frac{4}{5}n$. The value v_s is set to zero when s is below a threshold T and set to one when s is above T . The threshold T represents the minimum size of a coalition that can pass a law. The threshold T is integer and follows a uniform distribution $U(1, n)$.

Dataset airport: Airport games are also common in the study of the Shapley value and this dataset is inspired from a classically used instance from the literature (Castro, Gómez, & Tejada, 2009). In this game each player (a plane) has a cost c_i (the cost of building a large enough track for it to take off). The value of a coalition S is $v(S) = \max_{i \in S} c_i$. In this dataset, each player obtains a random integer cost from a uniform distribution $U(1, 100)$. An airport game can be modeled as a sum of symmetric games. Let us assume without loss of generality that the players are sorted by cost : $c_i \leq c_{i+1}$. There is one symmetric game v^i for each player i whose value function is $v^i(S) = c_i - c_{i-1}$ if S contains a player j with $j \geq i$ (and thus with a higher cost) otherwise $v^i(S) = 0$. Dummy players for that game are the players j with $j \leq i$. Note that this method to reformulate the problem corresponds to the classical way of obtaining an exact Shapley value for airport games.

6.2. Tested algorithms

We now detail which algorithms are compared in our numerical results. Note that on each instance, each algorithm is launched with four different sample budgets being equal to an^2 where a varies in $\{1, 4, 20, 100\}$ and n is the number of player of the instance.

We give first the algorithms that we re-implemented from the literature.

Algorithm kernel: This algorithm is based on sampling subsets and solving the quadratic program presented in Section 2.3. It is a re-implementation of the algorithm KernelSHAP.

Algorithm mc no stratif: This algorithm is based on sampling marginal contributions as presented in Section 2.1 (the sampling based on permutations is used). No stratification is used.

Algorithm mc position stratif: Similar to the previous algorithm except stratification is used with the event partition ("The drawn subset is of size k ") $_{0 \leq k \leq n-1}$.

Algorithm mc position stratif neyman: Similar to the algorithm *mc position stratif* except that the Neyman sample allocation is used as presented in Section 3.3. Note that this prevents us from sampling marginal contribution with permutation. Thus, we use the second method presented in Section 2.1.

Algorithm cc position stratif: This algorithm is based on sampling complementary contributions as presented in Section 2.2. Stratification is used with the event partition ("The drawn subset is of size k ") $_{0 \leq k \leq n-1}$. This stratification is mandatory for complementary contribution algorithm in order to obtain a properly converging estimator of the Shapley value.

Algorithm cc position stratif neyman: Similar to the algorithm *cc position stratif* except that the Neyman sample allocation is used. See (Zhang et al., 2023) for details of the sample allocation.

The two algorithms bellow are the algorithms that benefit from the new stratification techniques we introduced in Section 4.

Algorithm mc player stratif: Similar to the algorithm *mc no stratif* except stratification is used with the events "The drawn subset contains player j " for all j . The recursive stratification strategy presented in Section 4.2 is used to combine these events into an event partition.

Algorithm mc both stratif: Similar to the previous algorithm except that stratification is also used with the event partition ("The drawn subset is of size k ") $_{0 \leq k \leq n-1}$. In order to apply both stratifications, the flexible stratification paradigm presented in Section 4.1 is used.

6.3. Results and analysis

In this section we present the numerical results obtained during our experimental campaign and answer the questions asked at the beginning of Section 6. In order to evaluate the efficiency of an algorithm on an instance we use the mean normalized error. This metrics takes the mean over all

players of the following quantity.

$$\frac{|\hat{\phi}_i - \phi_i|}{|\phi_i| + \epsilon}$$

where ϕ_i is the true Shapley value for player i , $\hat{\phi}_i$ is the estimate returned by the algorithm and $\epsilon = 10^{-1}$. Note that this means that in our figures, the lower the curve the better the corresponding algorithm is performing. On each figure, one color corresponds to one algorithm. The main line represents the median over 20 instances sharing the same characteristics. The filled area around the main curve represents the 25% and 75% quantiles for the same 20 instances.

Comparison between estimators based on complementary contributions and marginal contributions : as discussed in Section 2.2, complementary contributions have to compensate their higher base variance by reusing their samples. As a result, when the difference in variance is low, complementary contributions are expected to be more efficient. This is the case for example with voting instances (see Figure 2a) where the variance of complementary contributions is only twice as high as the variance of marginal contributions (0.03 vs 0.015). However, in the dataset with monotone instances the variance of complementary contributions is 25 while the variance of marginal contributions is 0.15 which appears to be too much to compensate for (see Figure 2b). The best choice in this case really depends on the dataset. However, estimating the variance of both marginal and complementary contribution can help predicting which algorithm may work the best on a new dataset.

The next paragraph presents the impact of stratification. This is done only for algorithms using marginal contributions since stratification is mandatory to use it in complementary contributions algorithms for them to converge properly.

Impact of stratification for marginal contributions estimators : as shown in Figure 1, stratifying according to subset size is beneficial on most datasets (at least all of those we tried). As for stratification according to the presence of other players using the dynamic stratification technique

presented in Section 4.2, it is effective on datasets where some players are heavily coupled with others such as the "monotone sparse". Preliminary test also showed similar results in the instance "minimum spanning tree" from the literature (Castro, Gómez, & Tejada, 2009). However, in datasets where players are not coupled, stratifying according to the presence of other players provides a very mild benefit. Finally, when stratification according to the presence of other players is effective, it is even more effective to also stratify according to player position at the same time. This shows the effectiveness of being able to stratify according to several event partitions as described in Section 4.1.

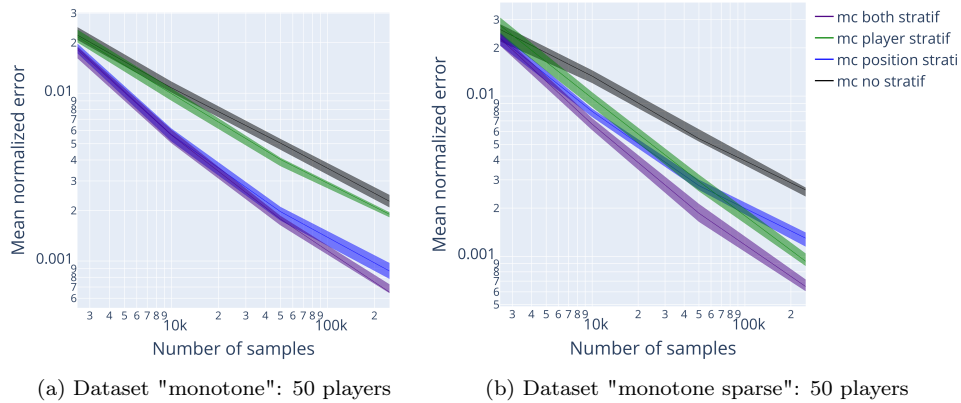


Figure 1: Comparison between several stratifications for marginal contributions based estimators

Impact of Neyman sample allocation : the sample allocation discussed in Section 3.3 has a positive impact on complementary contributions estimators in most of the datasets we tried. However, as shown in Figure 2, this is not always the case for marginal contributions. Although this may sound counter intuitive the reason is simple. We mentioned in Section 2.1 that not all ways of sampling were equivalent. Sampling using permutations creates twice as many samples for the same amount of value function evaluations as the other sampling methods. However, this method is not compatible (at least in a simple way) with sampling subsets of a fixed size. This means the estimator using neyman allocation starts with a sample deficit

compared to the other methods. This sometimes results with worse estimations. Also, we can see in Figure 2a that adding Neyman allocation does not have the same impact on low and high sample budgets. When the sample budget is too low, there is not enough budget to do the first phase of the Neyman allocation algorithms and evaluate the variances (see Section 3.3 for details). In this case, Neyman allocation has no benefits. However, when the sample budget is high, the variances can be properly evaluated and used in the Neyman allocation. Then the sample allocation can give enough benefit to compensate for the sample deficit created by not sampling using permutations.

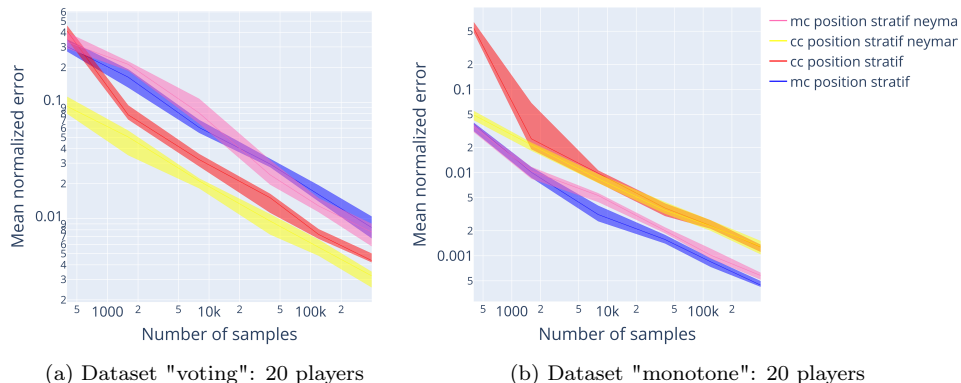


Figure 2: Comparison between estimators based on complementary contributions and marginal contributions

Comparison between marginal contributions vs KernelSHAP : on all dataset that we tested, the estimator KernelSHAP based on the quadratic program presented in Section 2.3 does not perform well compared to the marginal contribution estimator (see Figure 3). Our preliminary results also showed that the quadratic program does not benefit from stratification (for example, ensuring each subset size obtains the required number of samples).

Identification of the best algorithms: we provide in Figure 4 a comparison of the algorithms "mc position stratif", "mc both stratif" and "cc position stratif neyman" on all tested dataset for varying numbers of

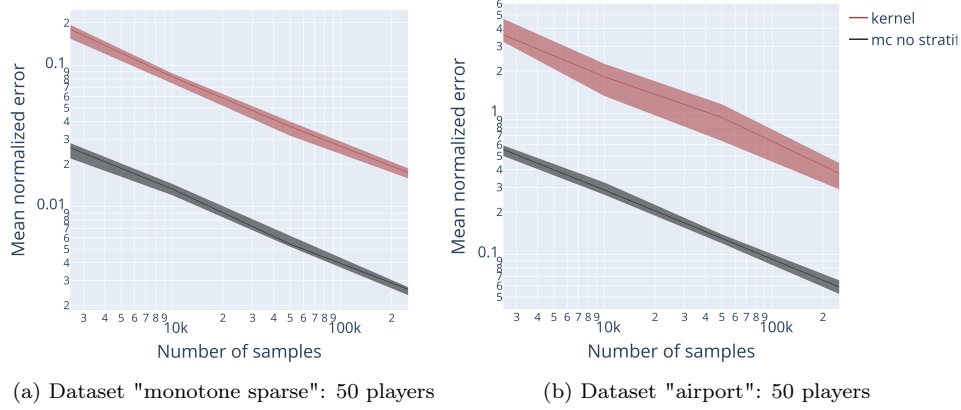
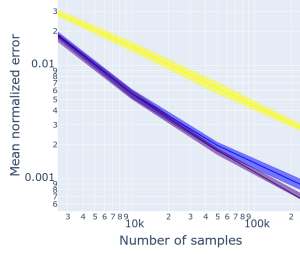


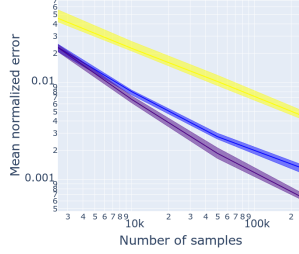
Figure 3: Comparison between marginal contributions and KernelSHAP

samples and players. Overall, it appears that the combination of "mc both stratif" and "cc position stratif neyman" dominate the other algorithms on almost all tested datasets. The best algorithm between the two is really dataset dependent but we remark that the best algorithm does not change when the number of players increases. Thus to determine the best algorithm on a particular dataset, one can test both algorithms on small games (where an exact computation is available) and use the better performing algorithm on larger games.

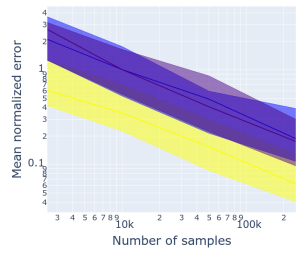
Remark: Note that when the sample budget is multiplied by 100 the precision of the results is multiplied by a factor around 10 and 20 depending on the algorithm and the dataset. This is coherent with the theoretical analysis proposed in Section 2 and 3.



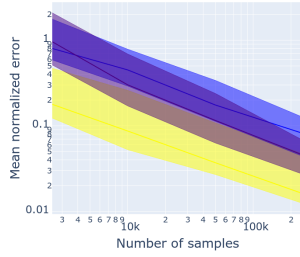
(a) Dataset "monotone": 50 players



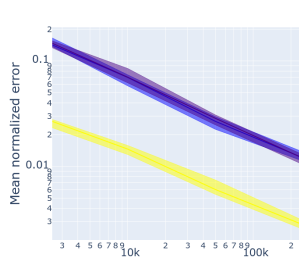
(b) Dataset "monotone sparse": 50 players



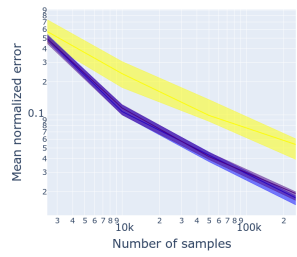
(c) Dataset "random": 50 players



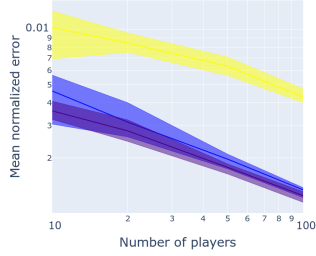
(d) Dataset "random sparse": 50 players



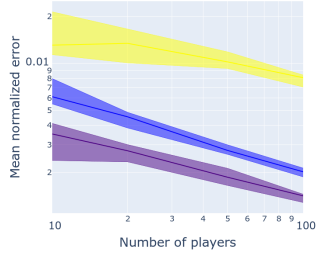
(e) Dataset "voting": 50 players



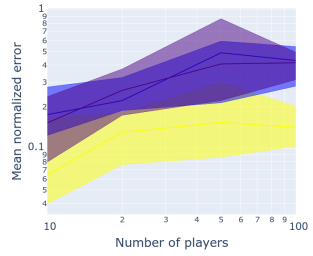
(f) Dataset "airport": 50 players



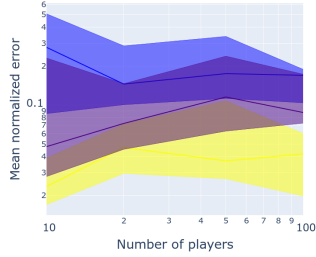
(g) Dataset "monotone": $20n^2$ samples



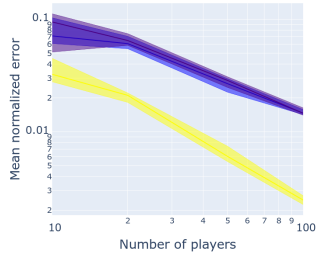
(h) Dataset "monotone sparse": $20n^2$ samples



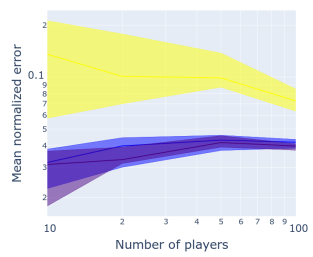
(i) Dataset "random": $20n^2$ samples



(j) Dataset "random sparse": $20n^2$ samples



(k) Dataset "voting": $20n^2$ samples



(l) Dataset "airport": $20n^2$ samples

Figure 4: Comparison of the best algorithms on all tested datasets for varying numbers of samples and players; same colors as before

7. Conclusion

In this work we studied how the Shapley value can be estimated using one of the most used techniques : sampling. We have shown that different algorithms from the literature can all be seen as estimating an expectation that coincides with the Shapley value. This interpretation helped us analyze when each algorithm should be more efficient by looking at the variance of the random variable whose expectation is estimated. This analysis was confirmed empirically in our experimental study.

Moreover, we propose a survey of the use of stratified sampling for strengthening the quality of the above mentioned algorithms. The theoretical analysis shows that stratified sampling helps removing bias from the sampling process. We used this knowledge to derive an algorithm that adapts the stratification performed to the result of the sampling process in order to obtain the best estimation possible. We also proposed a flexible method able to apply several stratification partitions at the same time which was shown to significantly enhance the performance on some datasets.

Finally we proposed a methodology to procedurally generate cooperative games. This enables the use of statistical tools when empirically comparing the different algorithms proposed in the literature. We conducted an experimental study comparing all the algorithms described in the paper. The study shows that two algorithms outperforms all the others on almost all the datasets. The best algorithm is dataset dependent but can be determined using small games as both algorithm seem to scale similarly with the number of players.

Acknowledgments

This document is the result of a research project funded by the research chair Retail Responsible and the industrial partner OneStock.

References

Burgess, M. A., & Chapman, A. C. (2021). Approximating the Shapley Value Using Stratified Empirical Bernstein Sampling. In *IJCAI*

- (pp. 73–81). Retrieved 2024-02-20, from <https://www.ijcai.org/proceedings/2021/0011.pdf>
- Castro, J., Gómez, D., & Tejada, J. (2009). Polynomial calculation of the shapley value based on sampling. *Computers & operations research*, 36(5), 1726–1730.
- Castro, J., Gómez, D., Molina, E., & Tejada, J. (2017, June). Improving polynomial estimation of the Shapley value by stratified random sampling with optimum allocation. *Computers & Operations Research*, 82, 180–188. Retrieved 2024-03-01, from <https://www.sciencedirect.com/science/article/pii/S030505481730028X> doi: 10.1016/j.cor.2017.01.019
- Castro, J., Gómez, D., & Tejada, J. (2009, May). Polynomial calculation of the Shapley value based on sampling. *Computers & Operations Research*, 36(5), 1726–1730. Retrieved 2024-03-01, from <https://www.sciencedirect.com/science/article/pii/S0305054808000804> doi: 10.1016/j.cor.2008.04.004
- Deng, X., & Papadimitriou, C. H. (1994). On the complexity of cooperative solution concepts. *Mathematics of operations research*, 19(2), 257–266.
- Kolpaczki, P., Bengs, V., Muschalik, M., & Hüllermeier, E. (2024). Approximating the shapley value without marginal contributions. In *Proceedings of the aaai conference on artificial intelligence* (Vol. 38, pp. 13246–13255).
- Maleki, S. (2015). *Addressing the computational issues of the shapley value with applications in the smart grid* (Unpublished doctoral dissertation). University of Southampton.
- Maleki, S., Tran-Thanh, L., Hines, G., Rahwan, T., & Rogers, A. (2014, February). *Bounding the Estimation Error of Sampling-based Shapley Value Approximation*. arXiv. Retrieved 2024-02-16, from <http://arxiv.org/abs/1306.4265> (arXiv:1306.4265 [cs])
- Mitchell, R., Cooper, J., Frank, E., & Holmes, G. (2022). Sampling permutations for shapley value estimation. *Journal of Machine Learning Research*, 23(43), 1–46.
- Myerson, R. B. (1991). *Game theory: Analysis of conflict*. Harvard University Press. Retrieved 2024-11-19, from <http://www.jstor.org/stable/j.ctvjjsf522>
- Okhrati, R., & Lipani, A. (2021). A multilinear sampling algorithm to estimate shapley values. In *2020 25th international conference on pattern recognition (icpr)* (pp. 7992–7999).
- Shapley, L. S., et al. (1953). A value for n-person games. *Princeton University Press*.

- Staudacher, J., & Pollmann, T. (2023). Assessing antithetic sampling for approximating shapley, banzhaf, and owen values. *AppliedMath*, 3(4), 957–988.
- Zhang, J., Sun, Q., Liu, J., Xiong, L., Pei, J., & Ren, K. (2023). Efficient sampling approaches to shapley value approximation. *Proceedings of the ACM on Management of Data*, 1(1), 1–24.

Appendix A. Proofs

Proposition 3. *Let us assume a given sample budget B and an event stratification $(A_k)_{1 \leq k \leq K}$. Then the number of samples x_k to allocate to the strata corresponding to A_k in order to minimize the variance of the estimator (16) should be proportional to $P(A_k)\sqrt{\mathbb{V}[X|A_k]}$:*

$$x_k = \frac{c_k B}{\sum_{1 \leq k \leq K} c_k}$$

where $c_k = P(A_k)\sqrt{\mathbb{V}[X|A_k]}$.

Proof. Let us denote x_k the number of samples allocated to A_k . Let us also denote $c_k = P(A_k)\sqrt{\mathbb{V}[X|A_k]}$. Then the allocation of samples $(x_k)_{1 \leq k \leq K}$ minimizing the variance of the estimator is given by the result of the following optimization problem.

$$\begin{aligned} \min \quad & \sum_{1 \leq k \leq K} \frac{c_k^2}{x_k} \\ \text{such that} \quad & \sum_{1 \leq k \leq K} x_k = B \end{aligned}$$

Using Lagrangian theory (*i.e.* the KKT conditions), we know that the solution of this problem satisfies the following set of equations.

$$\begin{aligned} \frac{\partial L}{\partial x_k} &= 0, \quad \forall k \in \{1, \dots, K\} \\ \frac{\partial L}{\partial \lambda} &= 0 \end{aligned}$$

where the lagrangian L is

$$L(\lambda, x_1, \dots, x_K) = \sum_{1 \leq k \leq K} \frac{c_k^2}{x_k} + \lambda \left(\sum_{1 \leq k \leq K} x_k - B \right)$$

The first set of equations leads to

$$\begin{aligned}\frac{\partial L}{\partial x_k} &= 0 \\ \iff -\frac{c_k^2}{x_k^2} + \lambda &= 0 \\ \iff x_k &= \frac{c_k}{\sqrt{\lambda}}\end{aligned}$$

while second set of equation leads to

$$\begin{aligned}\frac{\partial L}{\partial \lambda} &= 0 \\ \iff \sum_{1 \leq k \leq K} x_k - B &= 0 \\ \iff \sum_{1 \leq k \leq K} \frac{c_k}{\sqrt{\lambda}} &= B \\ \iff \sum_{1 \leq k \leq K} \frac{c_k}{B} &= \sqrt{\lambda}\end{aligned}$$

and thus finally by reinserting into the first set of equations

$$x_k = \frac{c_k B}{\sum_{1 \leq k \leq K} c_k}$$

which leads to the appropriate form.

□